

Studies in Architecture: Running From Legacy Inflexibility

Legacy systems at this large insurance company were chaotic, so the company designed a new architecture intended to be flexible enough to accommodate changes it might undergo in the future.

Core Topic

Software Infrastructure: Software
Architecture

Key Issues

How will business and technology trends
reshape architectural trade-offs during the
next five years?

What will be the critical success factors for
implementing enterprise architecture during
the next five years?

Nearly every large organization is trying to cope with the challenge of implementing new technologies, generally immature and untested, while maintaining its legacy systems, which are often brittle and inflexible to change. For an example of this, we looked at a large insurance company that decided to replace its legacy systems with a wholly new infrastructure, and here we evaluate its prospects for success.

Problem: The company's IMS-based infrastructure, while capable, reliable and manageable, is extremely inflexible to significant changes in functionality. There are three reasons for this inflexibility:

Corrupt Application Design — Examination of the legacy applications revealed portions of code with “if ... then” structures nested 10 layers deep. Understanding these highly complex logic structures requires an intimacy with the application that few, if any, in the organization possess. The amount of system analysis required to add new functionality to such applications is generally prohibitively expensive.

Poorly Documented Business Rules — The legacy applications are a codification of business processes and rules that have evolved over decades. Divining those business rules from the legacy systems would be nearly impossible, given the complexity of the systems and their poor level of documentation. Significant modification of systems would require a knowledge of the internal workings of those systems that no single person in the organization has.

Interapplication Spaghetti — The communication links between applications are chaotic and unmanageable, the result of 30 years of incrementally conceived and deployed links

GartnerGroup

Entire contents © 1998 by Gartner Group, Inc. All rights reserved. Reproduction of this publication in any form without prior written permission is forbidden. The information contained herein has been obtained from sources believed to be reliable. GartnerGroup disclaims all warranties as to the accuracy, completeness or adequacy of such information. GartnerGroup shall have no liability for errors, omissions or inadequacies in the information contained herein or for interpretations thereof. The reader assumes sole responsibility for the selection of these materials to achieve its intended results. The opinions expressed herein are subject to change without notice.

Note 1

New Architecture Stipulates That a New Applications Must Be

- "Headless" (i.e., not tightly coupled with its associated user interface logic).
- Componentized or componentizeable (i.e., will expose functionality through externally callable application programming interfaces).
- Not tightly bound with its own workflow, document management and output functionality.
- Able to externalize its business rules out of its procedural code.

Critical Success Factors/Lessons Learned

A flexible computing infrastructure is one that is tolerant of diversity in future application designs, and this company has done little to support heterogeneous application types. To promote flexibility, it could deploy a message broker. Message brokers relieve the "interapplication spaghetti" problem by facilitating communication among systems with different data syntax and semantics (see Note 2).

Note 2

Message Broker Functionality

A message broker removes the responsibility for negotiating interapplication communication from any single application system and places it with the system software sitting *between* application systems. This frees the application developer from needing to know the physical location of the target application or the native formats and protocols of the target systems. In addition, the following value-added services are often found.

Flow Control: Rules for determining the flow of messages among participating systems

Transformation: Contents of incoming messages are modified so that they are syntactically and semantically consistent with target applications

Warehousing: A persistent store of messages that flow across applications is kept (e.g., for non-repudiation)

between applications. Changes to any single application system require testing the entire environment. With one application that was evaluated for replacement, system analysts found nearly 4,500 interfaces to external systems. With this level of complexity, documenting the flow of data between applications is nearly impossible.

Objective: The objective is to develop an architectural plan that, if deployed, will enable greater flexibility for change than the current one. Management is particularly concerned with its ability to react to new competitive pressures (e.g., a new entrant that uses the Internet to deliver low-cost insurance) and changes in the regulatory regime (e.g., financial services deregulation).

Approach: The company has developed a plan for a service-oriented architecture that will enable end users to invoke shared business logic through a range of access channels (e.g., the Web, telephone, fax and E-mail). The planned infrastructure will use an object request broker to connect a distributed set of tightly integrated in-house and purchased functionality (business rules, workflow, document output and document management). For custom development, a suite of development tools will be constructed that will enable business rules to be stored in an external repository and modified without changing procedural code.

Results: While the company's new architectural plans take pains to maximize the degree of *integration* (e.g., shared workflow, and document output and management services) they do little to maximize the degree of *flexibility*. As a result, the architecture has not met its stated objectives because the proposed approach requires that new functionality (purchased or developed in-house) meet a strict set of requirements (see Note 1). However, when new business functionality is required by business units, speed of deployment will take precedence over architectural compliance. That is, *business units will deploy applications that do not fit into the highly integrated architecture*, and then will inevitably patch them together on an ad hoc basis. *The result will be more interapplication spaghetti.*

Bottom Line: The technology landscape is changing so quickly that IT infrastructures must be built to change rather than built to last. Architectures are brittle when they require developed and (especially) purchased functionality to conform and interoperate with particular design patterns, middleware, operating systems and databases. Organizations developing strategic plans for their IT infrastructures should look to concepts that maximize tolerance of heterogeneity. Message brokers are a good example of such a concept.

